

Formalizing Probability Theory using the Lean Interactive Theorem Prover

Oxford, January 5th, 2026

1. **Some history**
2. Short introduction to Lean
3. Discrete Probability in Lean
4. Brownian Motion in Lean



has started Lean in 2013 at Microsoft



Conclusion

Lean is an **efficient programming language** and **proof assistant**.

Lean is very extensible and is implemented in Lean.

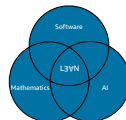
Lean proofs are maintainable, stable, and transparent.

Progress is accelerating with the Lean FRO: module system, new compiler, new proof automation, etc.

The Mathlib community is changing how math is done.

AI + Lean will have a huge impact on mathematics and software/hardware verification.

It is not just about proving but also understanding complex objects and proofs, getting new insights, and navigating through the “thick jungles” that are **beyond our cognitive abilities**.





promotes Lean since 2018

The rise of
formalism in
mathematics.

Kevin Buzzard

Live Lean
demo

Computer
theorem
provers

History

Formalising
statements

Recent history

The rise of formalism in mathematics.

K. Buzzard, Imperial College London

9th July, International Congress of Mathematicians



led the Liquid Tensor Experiment



Physics Mathematics Biology Computer Science Topics Archive

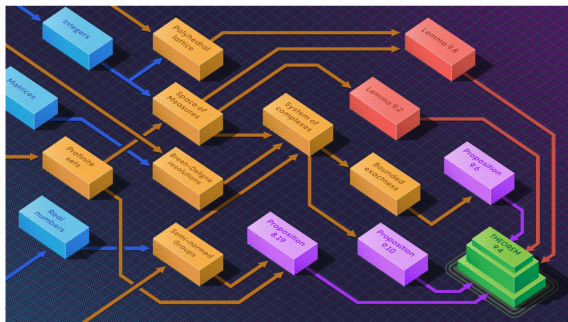


PROOF

Proof Assistant Makes Jump to Big-League Math



Mathematicians using the computer program Lean have verified the accuracy of a difficult theorem at the cutting edge of research mathematics.

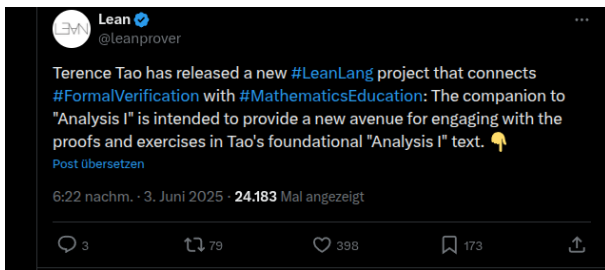


Somesh Valsaraj/Quanta Magazine, Johan Commelin

Terence Tao



leads collaborative efforts using Lean



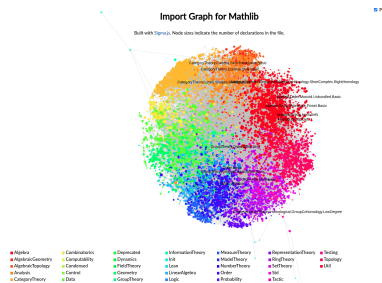


Three AI-systems won Gold medals (all with 5 out of 6 correct solutions):

- ▶ OpenAI: Solutions in natural language, general methods
- ▶ Google DeepMind: Solutions in natural language, special mathematical methods
- ▶ Harmonic: Solutions in Lean4, no human checks of solutions necessary.

Lean4

- ▶ Lean4 is a functional programming language.
- ▶ Today, Lean4 is developed by the Lean FRO with ≈ 20 people.
- ▶ Key features:
Meta-programming,
Interactive Theorem Proving.
- ▶ The largest **mathematical library** is mathlib with about $2 \cdot 10^6$ lines of code.



1. Some history
2. **Short introduction to Lean**
3. Discrete Probability in Lean
4. Brownian Motion in Lean

Lean4 demo

- ▶ Divisibility
- ▶ Unintuitive behavior
- ▶ Graphs

- ▶ Every *term* has a *type*, e.g.:
 $n : \mathbb{N}$ or $\alpha : \mathbf{Type}$ or
 $\text{negneg_of_classical } (p : \mathbf{Prop}) : (p \vee \neg p) \rightarrow (p \leftrightarrow \neg \neg p)$
- ▶ There is a hierarchy of types: **Type** 0 has type **Type** 1 (in order to avoid logical pitfalls).
- ▶ Every type can be constructed, e.g.:
inductive $\mathbb{N}$ where
| zero : $\mathbb{N}$
| succ (n : $\mathbb{N}$) : $\mathbb{N}$
The construction of a **Prop**-type is its proof.

- ▶ Lean4 uses *dependent types*. Example:

```
def Vec ( $\alpha$  : Type) ( $n$  :  $\mathbb{N}$ ) := { v : List  $\alpha$  // v.length = n }
```

Type Vec, which depends on the term $n : \mathbb{N}$

Errors in the length of a Vec are discovered at compile-time, not at run-time.

- ▶ Successful compilation of a lean file $\iff$ all proofs in that file are correct.
- ▶ The Curry-Howard isomorphism is described as *proofs-as-programs* und *propositions-as-types*. Example:
 $\forall x : \mathbf{Type}^*, x = x := \mathbf{fun} \ x \Rightarrow \mathbf{rfl}$

1. Some history
2. Short introduction to Lean
3. **Discrete Probability in Lean**
4. Brownian Motion in Lean

Discrete Probability

```
structure DiscreteMeasure ( $\alpha$  : Type*) where  
  weight :  $\alpha \rightarrow \mathbb{R}_{\geq 0\infty}$ 
```

```
@[coe]
```

```
noncomputable def toMeasure :
```

```
  DiscreteMeasure  $\alpha \rightarrow$  @Measure  $\alpha$  T :=
```

```
  fun  $\mu \mapsto$  sum (fun  $a \mapsto$  ( $\mu$ .weight  $a$ ) • @Measure.dirac  $\alpha$  T  $a$ )
```

```
structure DiscreteProbabilityMeasure ( $\alpha$  : Type*) where
```

```
  toDiscreteMeasure : DiscreteMeasure  $\alpha$ 
```

```
  hasSum_one : HasSum toDiscreteMeasure.weight 1
```

```
noncomputable def coin ( $p : \mathbb{R}_{\geq 0}$ ) ( $h : p \leq 1$ ) :
```

```
  DiscreteProbabilityMeasure Bool where
```

```
  toDiscreteMeasure :=
```

```
    (fun ( $b$  : Bool)  $\mapsto$  if  $b$  then ( $p : \mathbb{R}_{\geq 0\infty}$ ) else ( $1 - p : \mathbb{R}_{\geq 0\infty}$ ))
```

```
  hasSum_one := by
```

```
    simp [Summable.hasSum_iff ENNReal.summable,  $h$ ]
```

Discrete Probability

```
noncomputable def binom1 (p : ℝ≥0) (h : p ≤ 1) (n : ℕ) :  
  DiscreteProbabilityMeasure ℕ where  
  toDiscreteMeasure :=  
    ⟨fun k ↦ (p ^ k * (1 - p) ^ (n - k)) * (n.choose k)⟩  
  hasSum_one :=  
    by ...
```

```
noncomputable def binom3 (p : ℝ≥0) (h : p ≤ 1) (n : ℕ) :=  
  (sequence (List.replicate n (coin p h))).map (List.count true)
```

```
noncomputable def binom5 (p : ℝ≥0) (h : p ≤ 1) (n : ℕ) :=  
  do  
    let X ← sequence (List.replicate n (coin p h))  
    return X.count true
```

Discrete Probability

```
noncomputable def binom4 (p :  $\mathbb{R}_{\geq 0}$ ) (h : p ≤ 1) :  
  (n :  $\mathbb{N}$ ) → DiscreteProbabilityMeasure  $\mathbb{N}$   
  | 0 => pure 0  
  | n+1 => do  
    let S ← binom4 p h n  
    let X ← coin p h  
    return S + X.toNat
```

```
noncomputable def binom6 (p :  $\mathbb{R}_{\geq 0}$ ) (h : p ≤ 1) (n :  $\mathbb{N}$ ) :=  
do  
  let mut S := 0  
  for _ in List.range n do  
    let X ← coin p h  
    S := S + X.toNat  
  return S
```


Discrete Probability (Insights from formalization)

- ▶ Discrete probability theory is like probability theory without any measurability issues. (This makes discrete probability measures a lawful monad.)
- ▶ Using **do**-Notation requires use of monads, and make definitions more readable.
- ▶ The binomial distribution can be defined in several equivalent ways.

1. Some history
2. Short introduction to Lean
3. Discrete Probability in Lean
4. **Brownian Motion in Lean**

Brownian Motion (History of the project)

- ▶ In 2023, Rémy Degenne (INRIA Lille) and I formalized the Kolmogorov Extension Theorem:

A family $(P_J)_{J \subseteq_f I}$ of probability measures on E_J for $J \subseteq_f I$, is projective if for all $H \subseteq J \subseteq_f I$, the pushforward $(\pi_H^J)_ P_J = P_H$.*

Extension theorem: a projective family $(P_J)_{J \subseteq_f I}$ defines a unique measure E^I whose finite dimensional marginals are the P_J 's.

- ▶ The paper was rejected, the project was stuck, but we wanted to formalize Brownian Motion.
- ▶ Rémy suggested to make a [collaborative project](#). This worked very well and was finished in only a few weeks.

Brownian Motion (Construction overview)

- ▶ Implement characteristic functions
(in order to characterize Gaussian measures)
- ▶ Build multivariate Gaussians
(only Gaussians on $\mathbb{R}$ in Mathlib at the time)
- ▶ Prove that the Gaussian family with Covariance $s \wedge t$ exists
and is projective.
- ▶ Use the Kolmogorov extension theorem in order to have a
stochastic process with correct fdds.
- ▶ Implement the Kolmogorov-Chentsov continuity theorem in a
modern version for general index sets
(Krätschmer & Urusov, 2023).
- ▶ Use the Kolmogorov-Chentsov continuity theorem to get a
modification with continuous paths. continuous paths.

Brownian Motion 1 (Kolmogorov extension)

```
noncomputable def kolmogorovExtension {ι : Type*}  
  {α : ι → Type*} {P : ∀ J : Finset ι, Measure (Π j : J, α j)}  
  [∀ i, PseudoEMetricSpace (α i)] [∀ i, BorelSpace (α i)]  
  [∀ i, SecondCountableTopology (α i)] [∀ i, CompleteSpace (α i)]  
  (P : ∀ J : Finset ι, Measure (Π j : J, α j)) [∀ i, IsFiniteMeasure (P i)]  
  (hP : IsProjectiveMeasureFamily P) : Measure (Π i, α i) := ...
```

Brownian Motion 2 (Kolmogorov-Chentsov Theorem)

*/-- A stochastic process $X : T \rightarrow \Omega \rightarrow E$ on an index space T and a measurable space Ω with measure P is said to satisfy the Kolmogorov condition with exponents p, q and constant M if for all $s, t : T$, the pair $(X s, X t)$ is measurable for the Borel sigma-algebra on $E \times E$ and the following condition holds:
 $\int \omega, \text{edist } (X s \omega) (X t \omega) \wedge^p \partial P \leq M * \text{edist } s t \wedge^q$. -/*

structure IsKolmogorovProcess (X : T → Ω → E) (P : Measure Ω)
(p q : ℝ) (M : ℝ ≥ 0) : **Prop** where
measurablePair : ∀ s t : T, Measurable[_, borel (E × E)]
fun ω ↦ (X s ω, X t ω)
kolmogorovCondition : ∀ s t : T,
 ∫ ω, edist (X s ω) (X t ω) $\wedge^p \partial P \leq M * \text{edist } s t \wedge^q$
p_pos : 0 < p
q_pos : 0 < q

Brownian Motion 3 (Continuous modification)

*-- A stochastic process is called ****pre-Brownian**** if its finite-dimensional laws are those of a Brownian motion, see ``gaussianProjectiveFamily``. -/*

```
class IsPreBrownian (P : Measure  $\Omega$  := by volume_tac) : Prop where
  mk' ::
    hasLaw :  $\forall$  I : Finset  $\mathbb{R}_{\geq 0}$ , HasLaw (fun  $\omega \mapsto$  I.restrict (X ·  $\omega$ ))
      (gaussianProjectiveFamily I) P
```

```
noncomputable def gaussianProjectiveFamily (I : Finset  $\mathbb{R}_{\geq 0}$ ) :=
  multivariateGaussian 0 (brownianCovMatrix I) |>.map
    (MeasurableEquiv.toLp 2 (I  $\rightarrow \mathbb{R}$ )).symm
```

*/-- A stochastic process is called ****Brownian**** if its finite-dimensional laws are those of a Brownian motion, see ``IsPreBrownian``, and if it has almost-sure continuous paths. -/*

```
class IsBrownian (X) (P : Measure  $\Omega$  := by volume_tac) :
  Prop extends IsPreBrownian X P where
    cont :  $\forall^m \omega \partial P$ , Continuous (X ·  $\omega$ )
```

Brownian Motion 4 (The definition)

/-- If `h : IsPreBrownian X P`, then `h.mk X` is a continuous modification of `X`. -/

protected noncomputable def IsPreBrownian.mk (X)

[h : IsPreBrownian X P] : $\mathbb{R}_{\geq 0} \rightarrow \Omega \rightarrow \mathbb{R} :=$
h.exists_continuous_modification.choose

noncomputable

def brownian : $\mathbb{R}_{\geq 0} \rightarrow (\mathbb{R}_{\geq 0} \rightarrow \mathbb{R}) \rightarrow \mathbb{R} :=$
isPreBrownian_preBrownian.mk

Brownian Motion (Insights from formalization)

- ▶ Kolmogorov extension theorem does not require the same space E at all time points.
- ▶ Sometimes constants in paper proofs are wrong; they are easy to fix in Lean.
- ▶ Often, results can be generalized from metric spaces to pseudo-metric spaces (Kolmogorov extension, Kolmogorov-Chentsov).
- ▶ Measurability has to be taken more seriously when working in Lean.

universität freiburg

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡